GUIA DO ESTUDO DE CASO · MATERIAL DO ALUNO

Desenvolvimento de Aplicações Multicamadas

SaúdeCerta — Sistema de Gestão de Clínica

Turma NOITE · C795NL (terças)

IFPA · Tecnologia em Análise e Desenvolvimento de Sistemas | **Componente:** TEADS0347 – Desenvolvimento de Aplicações Multicamadas (67h) | **Docente:** Márcio Goes do Nascimento | **Período:** 2026.2

1. Apresentação e objetivos de aprendizagem
2. Contexto e problema (cenário)
3. Glossário de conceitos-chave
4. Escopo do sistema
5. Atores e personas
6. Requisitos funcionais
7. Regras de negócio
8. Requisitos não funcionais e cenários de qualidade
9. Casos de uso principais (fluxos)
10. Modelo de domínio
11. Arquitetura em camadas
12. Estilos e padrões aplicáveis
13. Arquitetura baseada em serviços (evolução no PF)
14. Decisões arquiteturais (ADRs preenchidas como exemplo)
15. Matriz de rastreabilidade (exemplo)
16. Atributos de qualidade e táticas
17. Roteiro por entrega
18. Rubrica detalhada
19. Ambiente, tecnologias e primeiros passos
20. Erros comuns a evitar
21. Referências de apoio

1 Apresentação e objetivos de aprendizagem

Este guia descreve o estudo de caso que você e sua equipe vão desenvolver ao longo de **todo o semestre**. Não se trata de construir um sistema enorme, e sim de **projetar, justificar e documentar** a arquitetura de um sistema real, aplicando a cada unidade da disciplina os conceitos vistos em aula. O mesmo sistema evolui do documento de visão (E1) até a implementação baseada em serviços e componentes (Projeto Final).

Ao concluir o estudo de caso, você deverá ser capaz de:

- Elicitar e organizar requisitos funcionais e não funcionais e transformá-los em **drivers arquiteturais**.
- Escolher e justificar **estilos e padrões arquiteturais** adequados ao problema.
- Projetar uma aplicação **multicamadas** (apresentação, negócio e persistência) em Java/Spring.
- Avaliar **tradeoffs** entre atributos de qualidade quanto a custo/benefício e registrar as decisões em ADRs.
- Garantir a **rastreabilidade** entre requisitos e arquitetura.
- Evoluir o sistema para uma **arquitetura baseada em serviços** e praticar reutilização por componentes e serviços.

Formato de trabalho: equipes de 3 a 4 integrantes; repositório Git obrigatório (código + documentação + ADRs versionados juntos); quatro entregas com prazos definidos no cronograma da turma.

2 Contexto e problema (cenário)

A **Clínica SaúdeCerta** é uma unidade de saúde de médio porte com cerca de vinte profissionais de diversas especialidades. Hoje a recepção controla a agenda em planilhas compartilhadas e os prontuários são mantidos em papel, guardados em pastas físicas.

Esse arranjo gera problemas recorrentes: **marcações conflitantes** no mesmo horário para o mesmo profissional; demora para localizar o histórico de um paciente que retorna; **retrabalho** na recepção quando o paciente remarca; e, principalmente, **risco no manuseio de dados sensíveis de saúde**, sem qualquer registro de quem acessou o quê.

A direção decidiu contratar o desenvolvimento de um **sistema web multicamadas** que digitalize o fluxo de atendimento, da marcação da consulta ao registro do prontuário e prescrição, preservando a privacidade dos dados e permitindo que o sistema cresça por módulos, sem reescrita, à medida que a clínica amadurece.

Por que este caso é interessante para arquitetura: ele coloca em tensão **segurança/privacidade** (dados de saúde) contra **usabilidade** (recepção ágil) e **desempenho** (abrir prontuário rápido) contra **auditabilidade** (registrar todo acesso). É exatamente esse tipo de conflito que exige decisões arquiteturais fundamentadas.

3 Glossário de conceitos-chave

Termo	Significado no contexto do caso
Camada (layer)	Divisão lógica de responsabilidades: apresentação, negócio/aplicação, domínio e persistência.

Tier	Divisão física/de implantação (ex.: servidor de aplicação e servidor de banco em máquinas distintas).
Driver arquitetural	Requisito (em geral não funcional) que mais influencia as decisões de arquitetura.
Atributo de qualidade	Característica mensurável do sistema: segurança, desempenho, disponibilidade, manutenibilidade etc.
Cenário de qualidade	Descrição testável de um atributo: fonte, estímulo, ambiente, resposta e medida da resposta.
ADR	Architecture Decision Record — registro curto de uma decisão arquitetural, suas alternativas e consequências.
RBAC	Role-Based Access Control — controle de acesso baseado em papéis (perfis).
Prontuário	Conjunto do histórico clínico do paciente: atendimentos, evoluções, prescrições e exames.

4 Escopo do sistema

Dentro do escopo: cadastro de pacientes, profissionais e especialidades; agenda e agendamento de consultas com verificação de conflito; atendimento com prontuário eletrônico; prescrições e solicitação de exames; autenticação e autorização por perfil; trilha de auditoria de acesso ao prontuário; relatórios gerenciais básicos.

Fora do escopo (proposital): faturamento e integração com convênios; integração com laboratórios externos para laudos; telemedicina/teleconsulta; assinatura digital com certificado ICP-Brasil. Esses itens ficam como **pontos de extensão futura** — úteis para você demonstrar que a arquitetura é modificável.

Módulos previstos

Módulo	Responsabilidade
Cadastros	Pacientes, profissionais e especialidades.
Agenda	Disponibilidade dos profissionais e marcação/remarcação/cancelamento de consultas.
Atendimento	Registro da consulta e evolução no prontuário eletrônico.
Prescrição/Exames	Emissão de prescrições e solicitação de exames vinculadas ao atendimento.
Segurança	Autenticação, perfis de acesso e auditoria.
Relatórios	Consultas por período, por profissional e por especialidade.

5 Atores e personas

Ator	Descrição	Principais objetivos
Recepcionista	Opera a linha de frente da clínica.	Marcar/remarcar consultas rapidamente, cadastrar pacientes, evitar conflitos de agenda.
Profissional de saúde	Médico(a) ou outro profissional que atende.	Consultar histórico, registrar atendimento, emitir prescrição e solicitar exames.
Paciente	Pessoa atendida pela clínica.	Ver seus agendamentos e seus dados; solicitar/confirmar consultas.
Administrador	Responsável técnico/gestão.	Gerenciar usuários e especialidades, acompanhar relatórios e auditoria.

Persona exemplo: Márcia, recepcionista. Atende telefone e balcão ao mesmo tempo, no horário de pico. Precisa marcar uma consulta em menos de um minuto e ser avisada na hora se o horário já estiver ocupado — usabilidade e resposta rápida são vitais para ela.

6 Requisitos funcionais

Cada requisito tem um identificador (para rastreabilidade), uma descrição e um critério de aceitação verificável.

ID	Requisito	Critério de aceitação
`RF01`	Cadastrar e manter pacientes.	Dado um paciente novo, ao informar nome, CPF e data de nascimento válidos, o sistema salva e passa a listá-lo na busca.
`RF02`	Cadastrar profissionais e vinculá-los a especialidades.	Um profissional pode ter uma ou mais especialidades; não é possível salvar sem ao menos uma.
`RF03`	Definir disponibilidade (agenda) do profissional.	O profissional/admin define dias e faixas de horário; o sistema gera os horários marcáveis.
`RF04`	Marcar consulta.	Ao escolher profissional, data e horário livres, a consulta é criada com status AGENDADA.
`RF05`	Remarcar/cancelar consulta.	Consulta AGENDADA pode ser remarcada ou cancelada; o horário anterior volta a ficar livre.
`RF06`	Impedir conflito de horário.	O sistema recusa marcação em horário já ocupado do mesmo profissional (ver RN02).
`RF07`	Registrar atendimento no prontuário.	Ao atender, o profissional registra queixa, evolução e conduta, vinculadas ao paciente e à consulta.
`RF08`	Emitir prescrição.	Uma prescrição contém um ou mais medicamentos (nome, dose, posologia) e fica vinculada ao atendimento.
`RF09`	Solicitar exames.	O atendimento pode gerar uma ou mais solicitações de exame com status de acompanhamento.

RF10	Consultar histórico do paciente.	O profissional visualiza, em ordem cronológica, atendimentos, prescrições e exames do paciente.
RF11	Autenticar usuários e aplicar perfis.	O acesso a cada função depende do perfil (recepção, profissional, admin, paciente).
RF12	Registrar auditoria de acesso ao prontuário.	Todo acesso/alteração de prontuário grava quem, quando e qual paciente (ver RNF02).
RF13	Gerar relatórios.	Consultas por período, por profissional e por especialidade, com totalizadores.

7 Regras de negócio

ID	Regra
RN01	Um paciente é identificado unicamente pelo CPF; não pode haver dois cadastros ativos com o mesmo CPF.
RN02	Um profissional não pode ter duas consultas AGENDADAS no mesmo horário.
RN03	Só é possível registrar atendimento para consultas cuja data já ocorreu ou é a do dia.
RN04	Prescrição e solicitação de exame só existem vinculadas a um atendimento.
RN05	O prontuário é somente-anexação: registros de atendimento não são apagados, apenas complementados.
RN06	Paciente só enxerga os próprios dados; recepção não acessa conteúdo clínico do prontuário.
RN07	Cancelamento de consulta exige que ela esteja com status AGENDADA.

8 Requisitos não funcionais e cenários de qualidade

Os RNFs são os principais **drivers** deste caso. Para cada um, escreva um cenário de qualidade testável.

ID	Atributo	Cenário de qualidade (fonte → estímulo → resposta → medida)
RNF01	Segurança/Privacidade	Um usuário sem perfil clínico tenta abrir um prontuário → o sistema nega e registra a tentativa → 100% das tentativas indevidas bloqueadas.
RNF02	Auditabilidade	Um profissional abre um prontuário → o sistema registra usuário, paciente, data/hora e ação → registro disponível para consulta do admin em até 1 min.
RNF03	Desempenho	Recepcionista busca um paciente pelo nome com a base em uso normal → resultado retornado em < 2 s no percentil 95.
RNF04	Disponibilidade	Durante o horário de funcionamento (12h/dia) → a agenda permanece operante → disponibilidade mensal ≥ 99% nesse período.
RNF05	Modificabilidade	Incluir o módulo de faturamento (futuro) → sem alterar o núcleo de agenda/prontuário → esforço estimado em até 5 dias/equipe.
RNF06	Usabilidade	Recepcionista marca uma consulta simples → conclui em ≤ 1 min e ≤ 5 cliques a partir da tela inicial.
RNF07	Segurança de credenciais	Senhas nunca são armazenadas em texto → uso de hash com sal (ex.: BCrypt) → verificado em revisão de código.

Tradeoffs para discutir na E2: auditoria completa síncrona pode penalizar o **desempenho** de escrita — vale registrar de forma assíncrona? Segurança forte (muitos perfis e confirmações) pode ferir a **usabilidade** da recepção — onde está o equilíbrio?

9 Casos de uso principais (fluxos)

UC01 — Marcar consulta (ator: Recepcionista)

1. Recepcionista busca o paciente (ou cadastra, se novo — RF01).
2. Seleciona a especialidade e o profissional.
3. O sistema mostra os horários livres da agenda (RF03).
4. Recepcionista escolhe data/horário e confirma.
5. O sistema valida conflito (RN02/RF06) e cria a consulta como AGENDADA (RF04).

Fluxo alternativo: horário ocupado → sistema informa e volta ao passo 3.

UC02 — Atender paciente (ator: Profissional)

1. Profissional abre a agenda do dia e seleciona a consulta.
2. O sistema registra o acesso ao prontuário (RF12/RNF02) e mostra o histórico (RF10).
3. Profissional registra queixa, evolução e conduta (RF07).
4. Se necessário, emite prescrição (RF08) e/ou solicita exames (RF09).
5. O atendimento é salvo e anexado ao prontuário (RN05).

10 Modelo de domínio

Principais entidades, atributos essenciais e relacionamentos. Refine na E2 conforme suas decisões.

Entidade	Atributos principais	Relacionamentos
Paciente	id, nome, cpf, dataNascimento, telefone, email	1 Paciente → N Consultas; 1 Paciente → 1 Prontuário
Profissional	id, nome, registroConselho, ativo	N Profissionais ↔ N Especialidades; 1 Profissional → N Agendas

Especialidade	id, nome	N ↔ N Profissional
Agenda/Disponibilidade	id, diaSemana, horaInicio, horaFim	N Agendas → 1 Profissional
Consulta	id, dataHora, status (AGENDADA/REALIZADA/CANCELADA)	N → 1 Paciente; N → 1 Profissional; 1 → 0..1 Atendimento
Atendimento	id, dataHora, queixa, evolucao, conduta	1 → 1 Consulta; 1 → N Prescrições; 1 → N SolicitaçõesDeExame
Prontuário	id, pacienteId	1 → 1 Paciente; agrega Atendimentos
Prescrição	id, dataEmissao	1 → 1 Atendimento; 1 → N ItensDePrescrição
ItemDePrescrição	id, medicamento, dose, posologia	N → 1 Prescrição
SolicitaçãoDeExame	id, tipoExame, status	N → 1 Atendimento
Usuário	id, login, senhaHash, perfil	1 → 0..1 Profissional / Paciente

11 Arquitetura em camadas

O ponto de partida é uma aplicação **multicamadas** com **arquitetura hexagonal** (o domínio no centro, isolado de infraestrutura por interfaces/portas). Comece como **monólito modular** bem fatiado por módulo de negócio.

Camada	Responsabilidade	Exemplos (Spring)
Apresentação	Expor a API e traduzir HTTP ↔ objetos; validação de entrada.	`@RestController`, DTOs, Bean Validation
Aplicação/Serviço	Orquestrar casos de uso, transações e regras de aplicação.	`@Service`, `@Transactional`
Domínio	Entidades, regras de negócio (RN01–RN07), invariantes.	Entidades ricas, portas (interfaces)
Persistência	Adaptadores de acesso a dados.	`@Repository`, Spring Data JPA
Transversal	Segurança, auditoria, tratamento de erros.	Spring Security, filtros/aspectos, `@ControllerAdvice`

Sugestão de estrutura de pacotes

```
br.edu.ifpa.saudecerta
├── agenda
│   ├── api          (Controllers, DTOs)
│   ├── application  (Services, casos de uso)
│   ├── domain       (Consulta, Agenda, regras)
│   └── infra        (Repositories JPA)
├── prontuario
│   ├── api  ├── application  ├── domain  └── infra
├── cadastro   (Paciente, Profissional, Especialidade)
├── seguranca  (Usuario, perfis, auditoria)
└── shared     (config, exceções, utilitários)
```

Boa prática: dependências apontam sempre para o domínio (regra da arquitetura hexagonal/limpa). A camada de persistência implementa interfaces definidas no domínio, e não o contrário.

12 Estilos e padrões aplicáveis

Padrão/Estilo	Onde aplicar no SaúdeCerta
Camadas (Layered)	Organização geral apresentação/serviço/domínio/persistência.
Hexagonal (Ports & Adapters)	Isolar o domínio clínico de banco, web e mensageria.
Repository	Acesso a Paciente, Consulta, Atendimento sem vazar JPA para o domínio.
Service	Casos de uso: marcar consulta, registrar atendimento.
DTO	Transferência entre API e cliente sem expor entidades.
RBAC	Autorização por perfil (recepção, profissional, admin, paciente).
Observer/Eventos	Publicar 'AtendimentoRegistrado' para gerar auditoria de forma assíncrona.

13 Arquitetura baseada em serviços (evolução no PF)

No Projeto Final, extraia pelo menos um serviço do monólito — por exemplo o **Serviço de Agendamento** ou o **Serviço de Prontuário** — expondo uma API REST bem definida e mantendo o restante como está. Documente o novo diagrama de contêineres (C4).

Exemplos de endpoints REST

Método	Rota	Descrição
POST	/api/pacientes	Cadastra um paciente (RF01).
GET	/api/pacientes?nome=	Busca pacientes por nome (RF01/RNF03).
GET	/api/profissionais/{id}/agenda?data=	Lista horários livres do profissional (RF03).
POST	/api/consultas	Marca consulta (RF04); valida conflito (RF06).
PATCH	/api/consultas/{id}	Remarca/cancela (RF05).

POST	/api/consultas/{id}/atendimento	Registra atendimento (RF07).
GET	/api/pacientes/{id}/prontuario	Histórico do paciente (RF10); gera auditoria (RF12).

Exemplo de payload — marcar consulta

```
POST /api/consultas
{
  "pacienteId": 1024,
  "profissionalId": 17,
  "dataHora": "2026-09-15T19:30:00"
}

201 Created
{
  "id": 5001,
  "status": "AGENDADA",
  "dataHora": "2026-09-15T19:30:00"
}
```

14 Decisões arquiteturais (ADRs preenchidas como exemplo)

Cada equipe deve produzir de 3 a 6 ADRs. Abaixo, dois exemplos completos no formato esperado.

ADR-001 — Monólito modular em vez de microsserviços no início

Contexto: A clínica é de médio porte e a equipe é pequena; microsserviços trariam complexidade de operação (deploy, rede, observabilidade) sem benefício imediato.

Decisão: Adotar um monólito modular, com módulos bem separados (agenda, prontuário, cadastro, segurança), pronto para extração futura de serviços.

Alternativas consideradas: Microsserviços desde o início; monólito não modularizado.

Consequências: (+) Simplicidade e velocidade de entrega; (+) fronteiras claras facilitam extrair um serviço no PF; (−) exige disciplina para não acoplar módulos.

ADR-002 — Auditoria de acesso ao prontuário de forma assíncrona por eventos

Contexto: RNF02 exige registrar todo acesso ao prontuário, mas RNF03 exige leitura rápida; auditoria síncrona a cada leitura penalizaria o tempo de resposta.

Decisão: Publicar um evento de domínio ao acessar o prontuário e processá-lo de forma assíncrona, gravando a trilha de auditoria fora do caminho crítico.

Alternativas consideradas: Gravar auditoria na mesma transação da leitura (síncrona).

Consequências: (+) Preserva o desempenho de leitura; (+) desacopla auditoria do caso de uso; (−) risco de pequena defasagem no registro; (−) exige tratar falhas do consumidor.

15 Matriz de rastreabilidade (exemplo)

A rastreabilidade liga requisito → decisão → componente → verificação. Preencha a matriz completa na E2.

Requisito	Decisão (ADR)	Componente/Tática	Como verificar
RNF02 Auditabilidade	ADR-002	AuditoriaService + evento AtendimentoAcessado	Teste que confirma o registro após acesso ao prontuário.
RNF01 Segurança	ADR (RBAC)	Spring Security + perfis	Teste de acesso negado para perfil sem permissão.
RF06 Conflito de horário	Regra RN02 no domínio	AgendaService.validarConflito()	Teste unitário com dois agendamentos no mesmo horário.
RNF03 Desempenho busca	Índice + paginação	PacienteRepository	Medição do tempo de resposta da busca por nome.

16 Atributos de qualidade e táticas

Atributo	Táticas arquiteturais aplicáveis
Segurança	Autenticação, autorização RBAC, hash de senha, princípio do menor privilégio, trilha de auditoria.
Desempenho	Índices de banco, paginação, consultas específicas (evitar N+1), cache de leitura quando fizer sentido.
Disponibilidade	Tratamento de exceções, timeouts, transações curtas, healthcheck (Spring Actuator).
Modificabilidade	Alta coesão e baixo acoplamento, módulos por domínio, interfaces/portas, DTOs.
Auditabilidade	Eventos de domínio, log estruturado, registro imutável de acesso.

17 Roteiro por entrega

E1 — Documento de Visão e Requisitos (3,0)

- ☐ Visão do produto e problema; lista de stakeholders e atores.
- ☐ Requisitos funcionais (RF) e não funcionais (RNF) identificados.
- ☐ Regras de negócio (RN) e drivers arquiteturais destacados.
- ☐ Escolha justificada do estilo arquitetural inicial.

E2 — Projeto Arquitetural (5,0)

☐ Diagramas C4: Contexto, Contêiner e Componente.
☐ Definição das camadas/componentes e do modelo de domínio.
☐ Cenários de qualidade para os principais RNFs.
☐ 3 a 6 ADRs com alternativas, tradeoffs e custo/benefício.
☐ Matriz de rastreabilidade requisitos ↔ arquitetura.

E3 — Protótipo em Spring Boot (3,0)

☐ Projeto Spring Boot com as camadas separadas (api/application/domain/infra).
☐ API REST do núcleo (cadastro de paciente, marcação de consulta, registro de atendimento).
☐ Persistência com Spring Data JPA e validação de entrada.
☐ Ao menos os testes das regras RN02 e RN05.

Projeto Final — Serviços e reutilização (5,0)

☐ Extração de pelo menos um serviço (ex.: Agendamento ou Prontuário) com API própria.
☐ Componentes reutilizáveis e injeção de dependências bem aplicada.
☐ Auditoria assíncrona funcionando (ADR-002).
☐ Documentação arquitetural consolidada e coerente com o código.
☐ Apresentação/defesa das decisões (tradeoffs e custo/benefício).

18 Rubrica detalhada

Critério	Peso	O que se espera
Requisitos e visão (E1)	3,0	RF/RNF/RN claros e completos; drivers e estilo bem justificados.
Modelagem e diagramas (E2)	2,0	C4 correto e coerente; modelo de domínio adequado.
Decisões e rastreabilidade (E2)	3,0	ADRs com tradeoffs reais; matriz consistente.
Implementação em camadas (E3)	3,0	Separação real de camadas; API REST correta; testes das regras.
Serviços e reutilização (PF)	3,0	Serviço extraído funcionando; reuso e DI bem aplicados.
Documentação e defesa (PF)	2,0	Documentação alinhada ao código; defesa consistente das decisões.
Avaliação contínua	4,0	2,0 em cada avaliação: participação, checkpoints e evolução.

Distribuição: E1 (3,0) + E2 (5,0) = N1; E3 (3,0) + PF (5,0) = N2; contínua 2,0 em cada. Média Final = (N1 + N2) / 2.

19 Ambiente, tecnologias e primeiros passos

- **Linguagem/plataforma:** Java 17+ e Spring Boot 3.x.
- **Dependências iniciais:** Spring Web, Spring Data JPA, Spring Security, Bean Validation, banco H2 (dev) e PostgreSQL (opcional).
- **Ferramentas:** IntelliJ IDEA/Eclipse/VS Code, Maven ou Gradle, Git/GitHub.
- **Modelagem:** draw.io, PlantUML ou Structurizr (para os diagramas C4).
- **Teste de API:** Postman ou Insomnia.

```
# criar projeto via start.spring.io (ou IDE)
# dependências: web, data-jpa, security, validation, h2

mvn spring-boot:run     # sobe a aplicação em http://localhost:8080
```

20 Erros comuns a evitar

- **Domínio anêmico:** colocar toda a regra no Service e deixar entidades só com getters/setters — perde-se a coesão.
- **Vazar JPA no domínio:** anotar entidades de domínio com detalhes de persistência espalhados por todo o código.
- **Controller gordo:** lógica de negócio dentro do `@RestController`.
- **Ignorar RNFs:** documentar só o que o sistema faz e esquecer segurança, desempenho e auditoria (os drivers deste caso).
- **ADR sem alternativa:** registrar a decisão sem mostrar o que foi descartado e por quê — some o tradeoff.
- **Documentação divergente do código:** diagramas que não correspondem ao que foi implementado no PF.

21 Referências de apoio

- BASS, CLEMENTS, KAZMAN. *Software Architecture in Practice*, 4. ed., 2021.
- RICHARDS, FORD. *Fundamentals of Software Architecture*, O'Reilly, 2020.
- MARTIN, R. C. *Arquitetura Limpa*, Alta Books, 2019.
- WALLS, C. *Spring in Action*, 6. ed., Manning, 2022.
- PRESSMAN; MAXIM. *Engenharia de Software*, 8. ed., 2016.
- Modelo C4: material de referência de Simon Brown (c4model.com).

Documento de apoio didático — os grupos podem ajustar escopo e decisões em acordo com o professor. As entregas E1 e E2 compõem a 1ª Avaliação (N1); E3 e o Projeto Final compõem a 2ª Avaliação (N2). Média Final = (N1 + N2) / 2.